SESSION 12:

Task 1:

SESSION 12 -TASK 1.
Spotifyartists Table g

use mydb;

create table SpotifyArtists

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

artist_id	name	followers
101	Taylor Swift	102000000
102	Ed Sheeran	115000000
103	Ariana Grande	93000000
104	Drake	81000000
105	The Weeknd	85000000

(artist_id INT PRIMARY KEY,

name VARCHAR(255) NOT NULL,

followers INT NOT NULL);

select * from spotifyartists;

Insert into Spotifyartists

values (101, 'Taylor Swift', 102000000),

(102, 'Ed Sheeran', 115000000),

(103, 'Ariana Grande', 93000000),

(104, 'Drake', 81000000),

(105, 'The Weeknd', 85000000),

(106, 'Billie Eilish', 88000000);

SESSION 12 -TASK 1.
Final output .csv

WITH TopArtists AS (

SELECT

```
    artist_id,
    name,
    followers
  FROM
    SpotifyArtists
  ORDER BY
    followers DESC
  LIMIT 3
)
SELECT
  artist_id,
  name,
  followers
```

SESSION 12 -TASK 1. Final output .csv

```
FROM
  TopArtists;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

artist_id	name	followers
102	Ed Sheeran	115000000
101	Taylor Swift	102000000
103	Ariana Grande	93000000

TASK 2:

SESSION 12 -TASK 2. flipkartorders .table

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

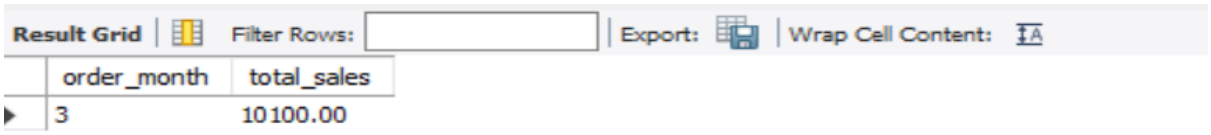
order_id	user_id	order_date	total_amount
101	1	2023-01-15	1200.00
102	2	2023-01-20	2500.00
103	3	2023-02-10	4500.00
104	1	2023-02-18	3100.00
105	4	2023-03-05	8900.00

```
use mydb;
CREATE TABLE FlipkartOrders (
   order_id INT PRIMARY KEY,
   user_id INT,
   order_date DATE,
   total_amount DECIMAL(10, 2)
);
select * from flipkartorders;
-- Insert Sample Data for 2023
INSERT INTO FlipkartOrders (order_id, user_id, order_date, total_amount) VALUES
(101, 1, '2023-01-15', 1200.00),
(102, 2, '2023-01-20', 2500.00),
(103, 3, '2023-02-10', 4500.00),
(104, 1, '2023-02-18', 3100.00),
(105, 4, '2023-03-05', 8900.00), -- Highest Month (March: 8900 + 1200 = 10100)
(106, 2, '2023-03-22', 1200.00),
(107, 5, '2023-04-12', 3400.00),
(108, 3, '2023-05-01', 5600.00),
(109, 2, '2023-11-11', 7800.00),
(110, 4, '2023-12-25', 4300.00);
```


SESSION 12 -TASK 2. final output .csv

```
WITH MonthlyTotals AS (
   SELECT
      EXTRACT(MONTH FROM order_date) AS order_month,
```

```
    SUM(total_amount) AS total_sales
  FROM
    FlipkartOrders
  WHERE
    EXTRACT(YEAR FROM order_date) = 2023
  GROUP BY
    EXTRACT(MONTH FROM order_date)
)
SELECT
  order_month,
  total_sales
FROM
  MonthlyTotals
ORDER BY
  total_sales DESC
LIMIT 1;
```



order_month	total_sales
3	10100.00

TASK 3:

Create the CalendarDays table structure C

REATE TABLE CalendarDays (id SERIAL PRIMARY KEY, day_date DATE NOT NULL); --

Populate the CalendarDays table using the recursive CTE

INSERT INTO CalendarDays (day_date) WITH RECURSIVE DateGenerator AS (SELECT CURRENT_DATE AS day_date, 1 AS level UNION ALL SELECT (day_date + INTERVAL '1 day')::DATE, level + 1 FROM DateGenerator WHERE level < 7)

SELECT day_date FROM DateGenerator; -- Verify the dataset SELECT * FROM CalendarDays;

WITH RECURSIVE CalendarDays AS (-- Anchor member: Starts with the current date SELECT CURRENT_DATE AS day_date, 1 AS level

UNION ALL –

Recursive member: Adds 1 day until reaching 7 days total SELECT (day_date + INTERVAL '1 day')::DATE, level + 1 FROM CalendarDays WHERE level < 7) –

Select the generated dates from the CTE SELECT day_date FROM CalendarDays;

TASK 4:

Result Grid | Filter Rows:

	id	name	city	rating
▶	1	Spice Hub	Mumbai	4.5
	2	Coastal Bite	Mumbai	4.2
	3	Urban Tadka	Mumbai	3.8
	6	Garden Grill	Bangalore	4.8
	7	Cafe Blossom	Bangalore	4.4


SESSION 12 -TASK 4. Zomatorestuarants


SESSION 12 -TASK 4. output .csv

use mydb;

Select * from Zomatorestaurants;

CREATE TABLE ZomatoRestaurants (

id INT PRIMARY KEY,

name VARCHAR(100),

city VARCHAR(50),

rating DECIMAL(2, 1));

-- Step 2: Insert Sample Data

INSERT INTO ZomatoRestaurants (id, name, city, rating) VALUES

(1, 'Spice Hub', 'Mumbai', 4.5),

(2, 'Coastal Bite', 'Mumbai', 4.2),

(3, 'Urban Tadka', 'Mumbai', 3.8),

(4, 'Delhi Bistro', 'Delhi', 3.9),

```
(5, 'Royal Curry', 'Delhi', 3.5),
(6, 'Garden Grill', 'Bangalore', 4.8),
(7, 'Cafe Blossom', 'Bangalore', 4.4);
```

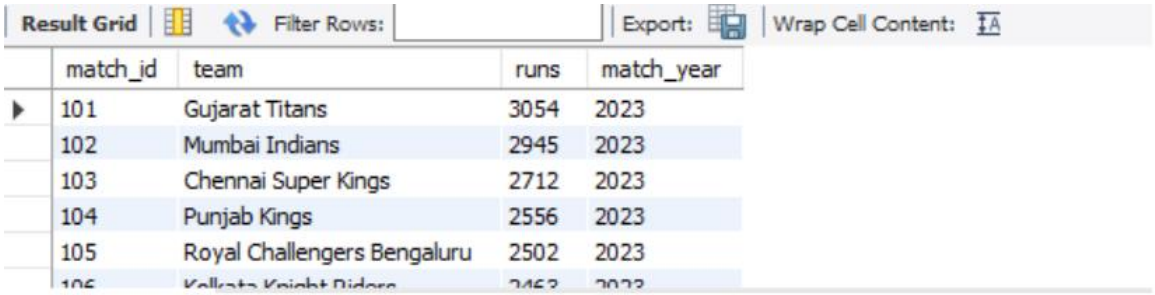
```
Select * from Zomatorestaurants;
```

```
WITH CityAvgRating AS (
  SELECT
    city,
    AVG(rating) AS avg_city_rating
  FROM ZomatoRestaurants
  GROUP BY city
  HAVING AVG(rating) > 4.0)
SELECT
  r.id,
  r.name,
  r.city,
  r.rating
FROM ZomatoRestaurants r
JOIN CityAvgRating c ON r.city = c.city;
```

TASK 5:



SESSION 12 -TASK 5
IPLMATCH TABLE .cs\



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

match_id	team	runs	match_year
101	Gujarat Titans	3054	2023
102	Mumbai Indians	2945	2023
103	Chennai Super Kings	2712	2023
104	Punjab Kings	2556	2023
105	Royal Challengers Bengaluru	2502	2023
106	Kolkata Knight Riders	2463	2023

```
 Use mydb;
 Select * from IPLMatches;
CREATE TABLE IPLMatches (
   match_id INT,
   team VARCHAR(50),
   runs INT,
   match_year INT
);
-- 2. Insert sample aggregations / total runs for IPL 2023 season
INSERT INTO IPLMatches (match_id, team, runs, match_year) VALUES
(101, 'Gujarat Titans', 3054, 2023),
(102, 'Mumbai Indians', 2945, 2023),
(103, 'Chennai Super Kings', 2712, 2023),
(104, 'Punjab Kings', 2556, 2023),
(105, 'Royal Challengers Bengaluru', 2502, 2023),
(106, 'Kolkata Knight Riders', 2463, 2023),
(107, 'Rajasthan Royals', 2419, 2023),
(108, 'Sunrisers Hyderabad', 2376, 2023),
(109, 'Lucknow Super Giants', 2354, 2023),
(110, 'Delhi Capitals', 2182, 2023);

Select * from IPLMatches;
```

SESSION 12 -TASK 5
FINAL OUTPUT.csv

```
WITH TeamRuns2023 AS (
  SELECT
    team,
    SUM(runs) AS total_runs
  FROM
    IPLMatches
  WHERE
    match_year = 2023
  GROUP BY
    team
)
SELECT
  team,
  total_runs
FROM
  TeamRuns2023
WHERE
  total_runs > 2000
ORDER BY
  total_runs DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

team	total_runs
Gujarat Titans	3054
Mumbai Indians	2945
Chennai Super Kings	2712
Punjab Kings	2556
Royal Challengers Bengaluru	2502